Python Practice Set: Dictionaries

Real-world tasks using everything you have learned so far.

Quick Concept Recap

A **dictionary** stores data as **key: value** pairs inside curly braces. You look up data by its key, not by position.

```
student = {"name": "Aman", "age": 17, "grade": "A"} student["age"] = 18 # update value
student["city"] = "Mumbai" # add new key del student["grade"] # delete a key
student.keys() -> dict_keys(["name", "age", "city"]) student.values() ->
dict_values(["Aman", 18, "Mumbai"]) student.items() -> key-value pairs (use in for
loops) "name" in student -> True # check if key exists
```

Rule of thumb: whenever you need to label your data (name, price, score, count) — reach for a dictionary.

Task 1 — Phone Contacts Book

You are building the contacts app on a basic phone. Each contact name maps to a phone number. The user can add contacts, search, update, delete, and view all.

What to build:

- Start with an empty dictionary called **contacts**.
- Run a **while True** menu loop showing these options every iteration: 1. Add contact 2. Search contact 3. Update number 4. Delete contact 5. View all 6. Exit
- **Option 1 — Add:** Ask for a name and number. If the name already exists in contacts (use the **in** keyword), print "Contact already exists". Otherwise add it and print "Contact saved."
- **Option 2 — Search:** Ask for a name. If found, print their number. If not, print "Contact not found."
- **Option 3 — Update:** Ask for a name. If it exists, ask for the new number and update it. If not found, print "Contact not found."
- **Option 4 — Delete:** Ask for a name. If it exists, delete it using **del** and print "Contact deleted.". If not, print "Contact not found."
- **Option 5 — View all:** If the dictionary is empty print "No contacts saved." Otherwise loop through **contacts.items()** and print each name and number neatly.
- **Option 6 — Exit:** Print "Goodbye!" and break.

Sample output:

```
1.Add 2.Search 3.Update 4.Delete 5.View all 6.Exit
Choice: 1
Name: Aman    Number: 9876543210
Contact saved.
```

```
Choice: 2
Search: aman
Aman -> 9876543210

Choice: 3
Name: Aman    New number: 9999999999
Updated successfully.

Choice: 5
Aman -> 9999999999
```

> Hint: use name.title() when saving so "aman" and "Aman" are stored the same way. Then search using the same .title() so lookups always match.

Task 2 — Canteen Order Bill

A college canteen has a fixed menu (dictionary). Students order items, and your program builds their bill.

What to build:

- Create a dictionary called **menu** with at least 6 items and their prices. Example: {"samosa": 15, "chai": 10, "dosa": 40, "juice": 30, "sandwich": 50, "coffee": 25}
- Print the full menu at the start using a for loop over menu.items().
- Create an empty dictionary called **order** to track what the student orders.
- Use a **while True** loop. Ask: "Enter item to order (or 'bill' to finish): "
- If the item is in the menu: check if it is already in the order dictionary. If yes, increase its quantity by 1. If no, add it to order with quantity 1. Print "Added [item]. Quantity: X"
- If the item is NOT in the menu, print "Item not available."
- When the user types **bill**: loop through order.items() and for each item calculate cost = quantity x menu price. Print each line as: "samosa x2 = 30". Keep a running total and print it at the end.
- If total is above 100, apply a 10% canteen discount and print the discounted total too.

Sample bill output:

```
---- Your Bill ----
samosa  x2  =  30
chai    x1  =  10
dosa    x1  =  40
--------------------
Total     : 80
No discount applied.
```

> Hint: order[item] = order.get(item, 0) + 1 is the cleanest way to increase quantity whether or not the item already exists in the order dictionary. .get(key, default) returns the default if the key is missing instead of crashing.

Task 3 — Word Frequency Counter

This is a classic real-world problem — apps like Google Docs and email spam filters count word frequencies constantly. Build a simple version.

What to build:

- Ask the user to type a sentence (a full line of text using input()).
- Convert the whole sentence to lowercase and split it into individual words using **.split()** — this gives you a list of words.
- Create an empty dictionary called **freq**.
- Loop through the list of words with a for loop. For each word, if it already exists in freq, add 1 to its count. If it does not, set its count to 1. (Tip: .get() makes this a one-liner.)
- After the loop, print each word and how many times it appeared.
- Then find and print the **most repeated word**. Use: most = max(freq, key=freq.get) — this finds the key with the highest value.
- Also print how many **unique words** the sentence had using len(freq).

Sample run:

```
Enter a sentence: the cat sat on the mat and the cat ran

the  -> 3
cat  -> 2
sat  -> 1
on   -> 1
mat  -> 1
and  -> 1
ran  -> 1

Most repeated word : the (3 times)
Unique word count  : 7
```

Task 4 — Student Report Card System

A teacher needs to enter subject marks for multiple students and generate a report card for any student on demand. Each student's record is itself a dictionary stored inside the main dictionary — this is called a **nested dictionary**.

```
Nested dictionary structure used here: students = { "Aman": {"Maths": 88, "Science": 75, "English": 90}, "Raj": {"Maths": 60, "Science": 55, "English": 70} }
```

What to build:

- Create an empty dictionary called **students**.
- Use a **while True** menu loop: 1. Add student 2. View report card 3. View all students 4. Exit

- **Option 1 — Add student:** Ask for the student's name. Then ask for marks in 3 subjects: Maths, Science, English (as integers). Store as a nested dictionary: students[name] = {"Maths": m, "Science": s, "English": e}
- **Option 2 — Report card:** Ask for a student name. If found, loop through their subject dictionary using .items() and print each subject and mark. Calculate their average (sum of values / 3). Use if/elif/else to assign a grade: 90+ is A, 75-89 is B, 60-74 is C, below 60 is D. Print the average and grade at the bottom of the report.
- **Option 3 — View all students:** Loop through students.keys() and print each name.
- **Option 4 — Exit:** Break.

Sample report card output:

```
---- Report Card: Aman ----
Maths    : 88
Science  : 75
English  : 90
--------------------------
Average  : 84.33
Grade    : B
```

> Hint: to get the sum of all values in a dictionary use sum(student_dict.values()), and len(student_dict) gives the number of subjects.

Final Task — Hotel Room Booking System

This task brings together everything: dictionary CRUD, nested dictionaries, loops, if/elif/else, string methods, and input/output.

Scenario:

A small hotel has 5 rooms. Each room has a type and price stored in one dictionary. Bookings (who is staying in which room) are stored in another dictionary. The receptionist uses your program to check availability, book rooms, and check out guests.

What to build:

- **Setup — Rooms dictionary:** Create a dictionary called **rooms** with 5 entries. Each key is a room number (101 to 105) and the value is another dictionary with "type" and "price". Example: rooms = {101: {"type": "Single", "price": 800}, 102: {"type": "Double", "price": 1400}, ...}. Fill all 5 rooms.
- **Setup — Bookings dictionary:** Create an empty dictionary called **bookings**. This will map room number to guest name when a room is booked.
- **Main loop — Menu:** 1. View all rooms 2. Book a room 3. Check out 4. View current bookings 5. Exit
- **Option 1 — View all rooms:** Loop through rooms.items(). For each room, print the room number, type, price, and whether it is "Available" or "Booked" (check if the room number is in the bookings dictionary).

- **Option 2 — Book a room:** Ask for the room number (int) and the guest's name. If the room number is not in rooms, print "Room does not exist." If it is already in bookings, print "Room already booked." Otherwise add it to bookings and print "Room [no] booked for [name]. Total charge: Rs.[price] per night."
- **Option 3 — Check out:** Ask for the room number. If it is in bookings, remove it using del and print "[guest name] checked out from Room [no]. Thank you!" If the room is not booked, print "Room is not currently booked."
- **Option 4 — Current bookings:** If bookings is empty print "No rooms booked." Otherwise loop through bookings.items() and print room number and guest name.
- **Option 5 — Exit:** Print total rooms booked = len(bookings) and break.

Sample view-all output:

```
Room 101 | Single  | Rs.800  | Available
Room 102 | Double  | Rs.1400 | Booked
Room 103 | Suite   | Rs.2500 | Available
Room 104 | Single  | Rs.800  | Available
Room 105 | Double  | Rs.1400 | Booked
```

Checklist — before you call it done

- Contacts book: searching for 'aman' finds 'Aman' — case-insensitive match works.
- Canteen bill: ordering the same item twice shows quantity 2, not two separate lines.
- Word counter: 'the' appearing 3 times shows 3, not 1.
- Report card: nested dictionary accessed correctly — students[name]["Maths"] works.
- Hotel system: a room that is booked shows 'Booked' in the view-all list.
- Hotel system: trying to book an already-booked room prints the correct error message.
- Hotel system: after checkout the room shows 'Available' again in the list.
- All menus loop until the user explicitly chooses Exit — no option crashes the program.